ELEC 475 – Lab 3: Pet Nose Localization

Post-Lab Report: Correction Model

Aytu Tuna – Student number 20354469

ELEC 475, Fall 2026 (individual submission, no partner)

1. Model

Both correction models take a 64 × 64 RGB patch, normalized with the ImageNet mean and standard deviation, and output two numbers (t_u, t_v). The corrected nose position is (û′, v̂′) = (c_u, c_v) + 64 · (t_u, t_v), where (c_u, c_v) is the first prediction rounded to the nearest pixel. All layers use the PyTorch default initialization. Every convolution is 3 × 3 with stride 1 and padding 1, and every max pooling is 2 × 2 with stride 2.

1.1 CorrectionNet (models/correction.py)

Layer	Definition	Output	Parameters
Input		3 × 64 × 64	0
Block 1	Conv(3→32) → BatchNorm(32) → ReLU → MaxPool	32 × 32 × 32	960
Block 2	Conv(32→64) → BatchNorm(64) → ReLU → MaxPool	64 × 16 × 16	18,624
Block 3	Conv(64→128) → BatchNorm(128) → ReLU → MaxPool	128 × 8 × 8	74,112
Block 4	Conv(128→128) → BatchNorm(128) → ReLU → MaxPool	128 × 4 × 4	147,840
Flatten		2,048	0
fc1	Linear(2048, 256) → ReLU	256	524,544
fc2	Linear(256, 2), no activation	2	514
Total			766,594

Table 1: CorrectionNet. The total matches the handout and the torchinfo summary.

1.2 CorrectionNetModified (models/correction.py)

I made two changes. Each block now has two convolutions before the pooling step instead of one (Conv → BatchNorm → ReLU → Conv → BatchNorm → ReLU → MaxPool, VGG style). This doubles the depth and gives each block a larger receptive field before the image is shrunk. I also added Dropout(p = 0.3) between fc1 and fc2. I kept the flatten of the 4 × 4 map instead of global average pooling, because the answer depends on where the nose is in the patch and average pooling would throw that away.

Layer	Definition	Output	Parameters
Input		3 × 64 × 64	0
Block 1	Conv(3→32), BN, ReLU, Conv(32→32), BN, ReLU, MaxPool	32 × 32 × 32	10,272
Block 2	Conv(32→64), BN, ReLU, Conv(64→64), BN, ReLU, MaxPool	64 × 16 × 16	55,680
Block 3	Conv(64→128), BN, ReLU, Conv(128→128), BN, ReLU, MaxPool	128 × 8 × 8	221,952
Block 4	Conv(128→128), BN, ReLU, Conv(128→128), BN, ReLU, MaxPool	128 × 4 × 4	295,680
Flatten		2,048	0
fc1	Linear(2048, 256) → ReLU → Dropout(0.3)	256	524,544
fc2	Linear(256, 2), no activation	2	514
Total			1,108,642

Table 2: CorrectionNetModified (BN = BatchNorm2d).

2. Training

Data. Only the 5,494 training images (train_noses.txt) are used. The 698 test images are never seen during training, and no setting was tuned on test results. There is no validation split; every model trains for the fixed 20 epochs and the last epoch is saved.

Patch sampling (utils/correction_dataset.py). Each time an image is loaded, a new patch centre is drawn: c_u = round(u + U(−40, 40)) and c_v = round(v + U(−40, 40)), where (u, v) is the true nose in original-image pixels. The 128 × 128 region centred on (c_u, c_v) is cut with img.crop() (anything outside the image is black) and resized to 64 × 64 with bilinear interpolation. The target is ((u − c_u) / 64, (v − c_v) / 64), so it stays inside about ±0.63. Since the centre is redrawn every epoch, the model sees a different patch of each image every epoch.

Augmentation (augmented runs only). With probability 0.5 the patch is flipped left to right and t_u becomes −t_u. The patch is centred on (c_u, c_v), so the flip only mirrors the offset. Then ColorJitter(brightness = 0.3, contrast = 0.3, saturation = 0.3) is applied; it does not move pixels, so the target is unchanged. The --no-augment runs skip both steps.

Optimization (train_correction.py, unchanged). MSE loss on the normalized offsets. Adam with learning rate 1e-3, betas (0.9, 0.999) and weight decay 1e-5. CosineAnnealingLR with T_max = 20, stepped once per epoch, so the learning rate goes from 1e-3 to near 0. Batch size 64, shuffled, 2 data loader workers, 20 epochs, seed 42 (Python, NumPy and PyTorch seeds plus a seeded shuffle generator). Training used an RTX 5070 Ti Laptop GPU with PyTorch 2.14.1 (CUDA 13.0) and took about 80 s per model.

Correction at test time (correct.py, unchanged). The first predictions of SnoutNet, SnoutNet-V and the mean ensemble (from the In-Lab predict.py run) are rounded, a patch is cut around each, the model runs once in eval mode, and the corrected point is centre + 64 · output. The exact commands are in commands.txt and the terminal output of every run is in logs/.

3. Outputs

3.1 Accuracy and cost tables (evaluate.py)

Errors are Euclidean distances in original-image pixels over the 698 test images. "corr." is CorrectionNet, "mod. corr." is CorrectionNetModified, and "(aug)" means trained with augmentation.

Model	Min	Mean	Median	Max	Std
Baseline (mean)	0.6	79.7	70.0	848.2	56.1
SnoutNet	0.9	57.0	44.3	1084.5	57.4
SnoutNet-A	1.8	44.8	36.9	690.2	39.6
SnoutNet-V	0.4	12.9	8.7	175.5	17.3
Ensemble (mean)	0.7	31.2	23.5	597.3	33.7
Ensemble (median)	0.4	29.4	20.2	676.3	38.0
SnoutNet + corr.	0.3	43.7	20.3	1064.6	61.2
SnoutNet + corr. (aug)	0.7	43.6	20.2	1062.4	61.3
SnoutNet + mod. corr.	1.0	45.4	24.3	1072.7	61.1
SnoutNet + mod. corr. (aug)	0.4	44.6	22.8	1080.2	61.1
SnoutNet-V + corr.	0.1	12.0	7.6	202.0	17.2
SnoutNet-V + corr. (aug)	0.1	12.1	7.7	201.2	17.2
SnoutNet-V + mod. corr.	0.1	10.9	6.7	198.2	17.1

SnoutNet-V + mod. corr. (aug)	0.3	11.0	6.7	196.3	16.9
Ensemble (mean) + corr.	0.2	20.4	10.5	598.4	34.4
Ensemble (mean) + corr. (aug)	0.4	20.5	10.1	594.7	34.6
Ensemble (mean) + mod. corr.	0.1	21.0	10.3	588.9	34.3
Ensemble (mean) + mod. corr. (aug)	0.0	20.7	10.2	596.3	34.4

Table 3: Accuracy (pixels). Rows 1 to 6 are the In-Lab results.

Model	**Params**	**ms / image**
Baseline (mean)	0	0.0
SnoutNet	5,617,794	0.6
SnoutNet-A	57,012,034	1.2
SnoutNet-V	134,268,738	4.3
Ensemble (mean)	196,898,566	6.1
Ensemble (median)	196,898,566	6.1
SnoutNet + corr.	6,384,388	2.1
SnoutNet + corr. (aug)	6,384,388	1.8
SnoutNet + mod. corr.	6,726,436	2.1
SnoutNet + mod. corr. (aug)	6,726,436	1.6
SnoutNet-V + corr.	135,035,332	5.0
SnoutNet-V + corr. (aug)	135,035,332	5.1
SnoutNet-V + mod. corr.	135,377,380	5.8
SnoutNet-V + mod. corr. (aug)	135,377,380	5.9
Ensemble (mean) + corr.	197,665,160	7.3
Ensemble (mean) + corr. (aug)	197,665,160	7.3
Ensemble (mean) + mod. corr.	198,007,208	7.4
Ensemble (mean) + mod. corr. (aug)	198,007,208	7.2

Table 4: Cost. The parameters and time of a corrected model include the first model(s). Time is for batch size 1 on the GPU. The same architecture can differ by about 0.5 ms between runs (for example 2.1 and 1.8 ms), so small time differences are noise.

3.2 Cumulative error distribution

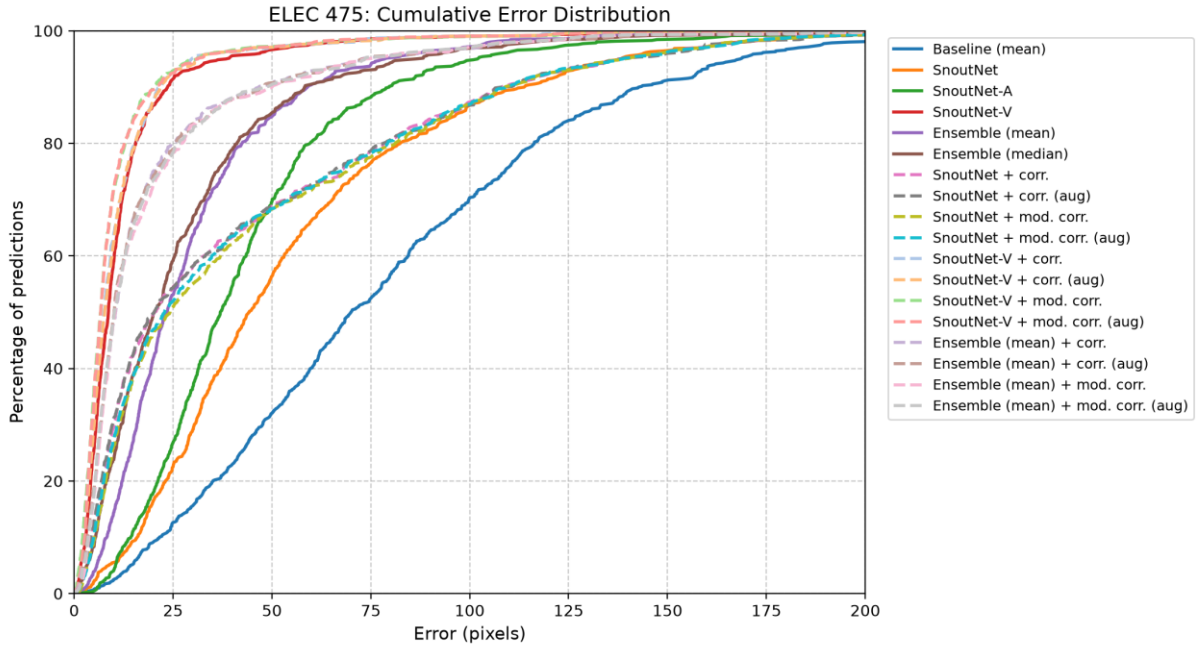


Figure 1: plots/ced.png. Dashed lines are corrected predictions.

3.3 Best and worst predictions



Figure 2: SnoutNet (plots/best_worst_snoutnet.png). Green circle: label, red cross: prediction.



Figure 3: SnoutNet + CorrectionNet (plots/best_worst_snoutnet_correction_noaug.png). Green circle: label, red cross: prediction.



Figure 4: SnoutNet + CorrectionNet with augmentation (plots/best_worst_snoutnet_correction_aug.png). Green circle: label, red cross: prediction.

3.4 Loss curves

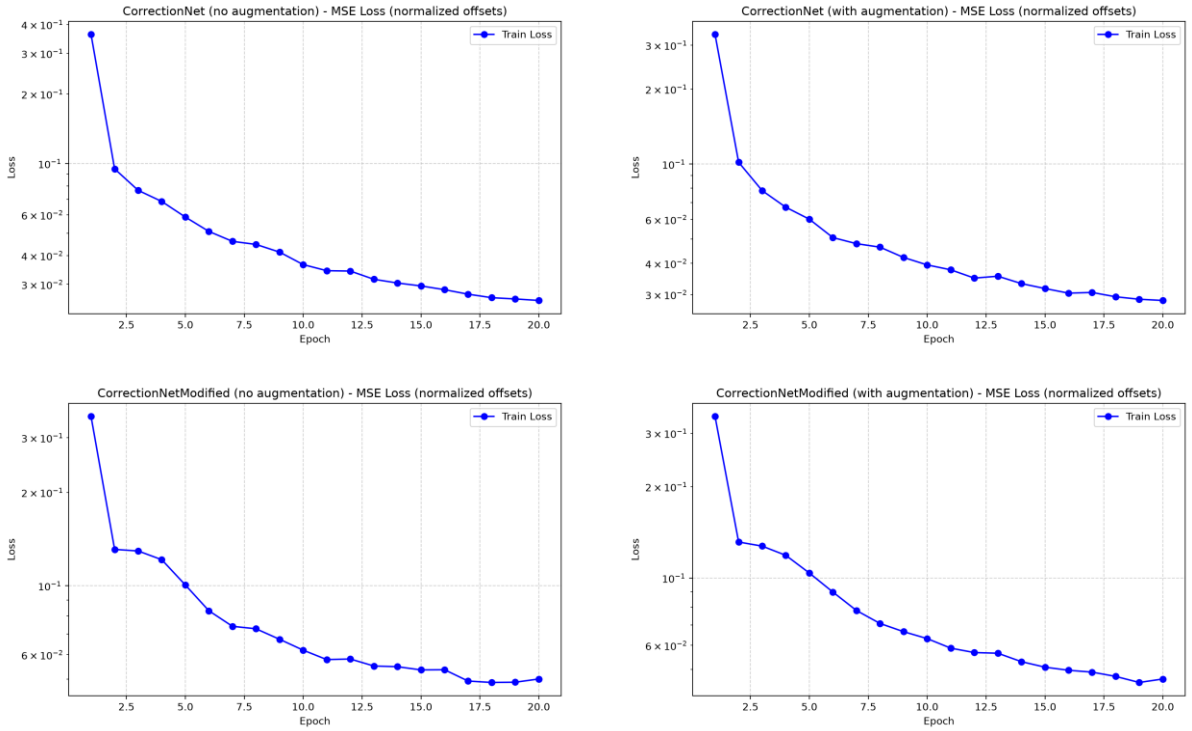


Figure 5: Training loss (log scale) of CorrectionNet without and with augmentation (top) and CorrectionNetModified without and with augmentation (bottom). All four fall steadily. The small bumps (largest at epoch 20 of the modified model) come from dropout and the random patches, and none of the curves flattens out early, so all four trained properly.

Final training loss and mean offset error (from logs/): CorrectionNet 0.0255 / 11.8 px without and 0.0284 / 12.4 px with augmentation; CorrectionNetModified 0.0501 / 16.6 px without and 0.0465 / 16.1 px with

augmentation. The modified numbers are measured with dropout on. Measured in eval mode on one fixed set of training patches, its error is 12.8 and 12.3 px, close to CorrectionNet.

4. Analysis

4.1 Failure cases

The worst images are almost the same for all three models (Egyptian_Mau_165, Egyptian_Mau_146, american_bulldog_217, scottish_terrier_109, newfoundland_116). In these the pet is not facing the camera and its head is not where it usually is: the cat or dog is lying on its side with the head at one edge, or has its head down sniffing the floor, or is small in a big scene next to a person. newfoundland_116 is also a black dog on a dark floor, so the face has almost no contrast. In all of these the prediction lands in the middle of the body, close to where noses usually are in the training set. Most training photos show a face from the front near the centre, so the models learn that habit. The whole image is also squashed to 227 × 227, so a nose that is far from the centre is only a few pixels and is easy to miss. Pixel errors also grow with image size: Egyptian_Mau_165 is 2560 × 1920, so the same mistake costs 1,084 px there.

4.2 Pretraining

SnoutNet trained from scratch reached 57.0 px mean error, SnoutNet-A 44.8 px and SnoutNet-V 12.9 px, against 79.7 px for always guessing the average nose position. There are only about 5,500 training images. That is not enough to learn good features from random weights, so SnoutNet ends up close to the average guess on hard images. ImageNet has 1.2 million images, including over 100 dog breeds and several cats, so the pretrained layers already find edges, fur, eyes and animal faces. Fine-tuning only has to learn where the nose is relative to those features. VGG16 does better than AlexNet because it is deeper, uses small 3 × 3 filters and keeps more spatial detail in its early layers.

4.3 Ensembles

The mean ensemble (31.2 px) and median ensemble (29.4 px) beat SnoutNet and SnoutNet-A but are much worse than SnoutNet-V alone (12.9 px), while costing the most. Averaging only helps when the members are about equally good and make different mistakes, so their errors cancel. Here SnoutNet-V is more than three times as accurate than the others, so the two weaker models pull its good answer away. The median does a little better because one bad member has less pull. I would expect an ensemble to beat its best member when the members have similar accuracy and errors that are not correlated, for example three VGG16 models trained with different seeds or augmentations, or a weighted average that trusts SnoutNet-V more.

4.4 Correction model

Yes, it helps. With CorrectionNet (no augmentation) the mean error went from 57.0 to 43.7 px for SnoutNet (median 44.3 to 20.3), from 31.2 to 20.4 px for the mean ensemble (median 23.5 to 10.5) and from 12.9 to 12.0 px for SnoutNet-V (median 8.7 to 7.6). It adds 0.77 M parameters and about 1 ms per image. Table 5 splits the test images by how far off the first prediction was.

First error	SnoutNet	SnoutNet-V	Ensemble (mean)

0–10 px	38: 5.6 → 7.9	411: 5.5 → 7.1	100: 6.6 → 7.3
10–30 px	165: 21.1 → 9.0	239: 15.5 → 11.9	345: 19.5 → 9.8
30–57 px	240: 42.1 → 19.1	29: 40.9 → 28.6	175: 40.6 → 19.3
> 57 px	255: 102.0 → 94.8	19: 95.4 → 93.1	78: 93.5 → 86.7

Table 5: Number of test images, and mean error before → after CorrectionNet (no augmentation), grouped by the error of the first prediction. 57 px is the largest training offset (40v2).

The correction works well when the first guess is between 10 and 57 px off: it cuts the error by half or more. It cannot help when the first guess is further than that. The model was never trained on larger offsets, and past 64 px the nose is not in the patch at all. This is 37% of SnoutNet's predictions, so its mean is still 43.7 px and its worst images are unchanged (Figures 2 to 4). It also slightly hurts guesses that are already within 10 px, because the correction model has its own error of about 7 to 8 px (the patch is shrunk by 2, so one patch pixel is two image pixels). This is why SnoutNet-V gains the least: 59% of its predictions are already within 10 px. The mean ensemble gains the most relatively (35%), since 74% of its predictions are 10 to 57 px off, which is exactly the range the correction model was trained for. Even corrected, the ensemble and SnoutNet stay behind SnoutNet-V alone.

4.5 Augmentation

Flip and colour jitter made almost no difference: 43.7 vs 43.6 px for SnoutNet, 12.0 vs 12.1 px for SnoutNet-V and 20.4 vs 20.5 px for the ensemble. For the modified model it helped a little for SnoutNet (45.4 to 44.6 px) and the ensemble (21.0 to 20.7 px), which is about the size of run-to-run noise. Augmentation is a fix for overfitting, and these models do not overfit. The random patch centre already gives a new patch every epoch, so the model never sees the same input twice. In eval mode CorrectionNet's error is 11.9 px on training patches and 12.1 px on test patches sampled the same way, so there is no gap to close. Flipping adds little because pet faces are close to symmetric, and colour jitter does not change where the nose is. With augmentation the task is a bit harder, so the final training loss was a little higher (0.0284 against 0.0255).

4.6 Modified correction model

I doubled the convolutions in each block and added dropout (Section 1.2), going from 0.77 M to 1.11 M parameters. The result was mixed. It improved SnoutNet-V from 12.0 to 10.9 px mean and 7.6 to 6.7 px median, the best result in the lab, and raised the share of SnoutNet-V predictions within 10 px from 64% to 72%. But it was slightly worse for SnoutNet (43.7 to 45.4 px) and the ensemble (20.4 to 21.0 px).

The reason shows up when the patch error is split by offset size. On test patches with the nose less than 15 px from the centre, the modified model's error is 6.8 px against 7.7 px. With the nose 45 to 60 px away it is 22.6 px against 18.4 px. The extra layers help it place the nose precisely when it is close, but it learned large offsets less well. A deeper network with dropout learns more slowly, and its loss was still falling over the last epochs (0.0536 at epoch 15 to 0.0501 at epoch 20 without augmentation), so 20 epochs were probably not enough. SnoutNet-V's first guesses are mostly close, so it gains. SnoutNet's and the ensemble's are often far, so they lose a little. Training longer, or using the original model for the first pass and the modified one for a second pass, would likely help. All runs use one seed, so differences under about 1 px between corrected rows may be noise.